

Milestone 1 — Computer Architecture

Design of a Vending Machine

Vy-Luong
rev 1.0.1

March, 2026

Abstract

The first milestone in the course involves designing a vending machine using SystemVerilog. Students are expected to adhere to specific coding guidelines and submit reports.

If you come across any errors or have suggestions for improving this document, please email the TA: vyluong05092003@hcmut.edu.vn with the subject [CA203 FEEDBACK].

1 Objective

- Understand coding guidelines.
- Review basic logic design and FSM concepts.
- Design a Vending Machine using SystemVerilog.

2 Coding Guidelines

When you're coding, it's important to remember that you'll be revisiting your code multiple times for various purposes like debugging and enhancement. Keeping your code clean will save you a significant amount of time. On the other hand, a messy code file can be visually overwhelming and mentally disruptive, leaving you wondering what you've done.

In this course, it is highly recommended to adhere to the coding guidelines provided in [this link](#).

3 Vending Machine Specification

Vending Machine is a dispenser machine that receives coins or bills and dispenses soft drinks or snacks. A simple vending machine controller is described in Figure 1

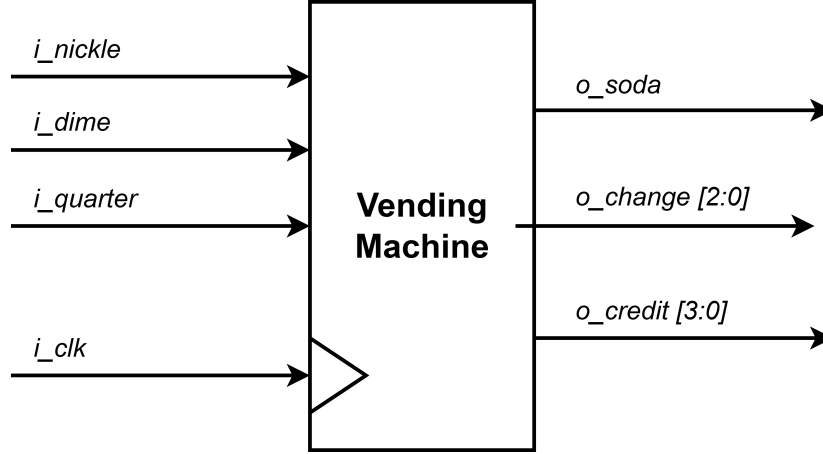


Figure 1: Vending machine top module

Table 1: Vending machine input and output signals

Signal	Width (bits)	Type	Description
i_clk	1	Input	System clock for the vending machine logic
i_nickle	1	Input	High when a nickel is inserted
i_dime	1	Input	High when a dime is inserted
i_quarter	1	Input	High when a quarter is inserted
o_soda	1	Output	High when a soda is dispensed
o_change	3	Output	Indicates the change returned after dispensing
o_credit	4	Output	Current accumulated credit in the machine
o_error	1	Output	High when more than one coin input is high in the same cycle

The vending machine accepts three types of coins: nickel, dime, and quarter. However, the system is designed to accept only one at a time. When a coin is inserted, the input is registered by the system and processed in the next clock cycle.

The value corresponding to each coin type is shown in Figure 2.

Coin	Value
Nickle	5 ¢
Dime	10 ¢
Quarter	25 ¢

Table 2: Type of coins

The price of a soda is 20¢. When the total value of the inserted coins is equal to or greater than 20¢, the vending machine dispenses a soda. This event is indicated by the output signal `o_soda`.

If the total inserted value exceeds the soda price, the vending machine must also return the appropriate change through the `o_change` output signal. The signal `o_change` is a 3-bit output, where each value represents a specific amount of returned change. The encoding of this signal is defined in Table 2.

<code>o_change [2:0]</code>	Description
000	The change is 0 ¢
001	The change is 5 ¢
010	The change is 10 ¢
011	The change is 15 ¢
100	The change is 20 ¢
101	<i>Reserved</i>
110	<i>Reserved</i>
111	<i>Reserved</i>

Table 3: `o_change` signal encoding

In addition to the `o_change` signal, a 4-bit output signal `o_credit` is used to represent the current accumulated value of coins inserted into the vending machine. This signal reflects the total credit available before or at the moment a soda is dispensed.

The `o_soda` signal is asserted when the value of `o_credit` is greater than or equal to 20¢, indicating that a soda should be dispensed. If the accumulated credit exceeds the soda price, the extra amount is returned as change through the `o_change` output, which is expected to be $\text{o_change} = \text{o_credit} - 20\text{¢}$.

The encoding of the `o_credit` signal is defined in Table 3.

<code>o_credit [3:0]</code>	Description
0000	The change is 0 ¢
0001	The change is 5 ¢
0010	The change is 10 ¢
0011	The change is 15 ¢
0100	The change is 20 ¢
0101	The change is 25 ¢
0110	The change is 30 ¢
0111	The change is 35 ¢
1000	The change is 40 ¢
1001	<i>Reserved</i>
...	...
1111	<i>Reserved</i>

Table 4: `o_credit` signal encoding

In practice, the vending machine is not expected to receive multiple coins simultaneously within the same clock cycle. However, due to possible hardware input conditions, more than one coin input signal may be asserted at the same time. Such a condition is considered an invalid operation.

When this occurs, the system asserts an `o_error` flag to indicate the faulty behavior. To prevent an incorrect transaction, the vending machine does not dispense a soda, and both the change and credit outputs are reset to 0¢. This allows the system to safely return to its initial state.

In this example, the system receives two nickels followed by a dime from the customer, resulting in a total credit of exactly 20¢. In the subsequent clock cycle, the vending machine dispenses a soda and no change is returned (`o_change` = 0¢).

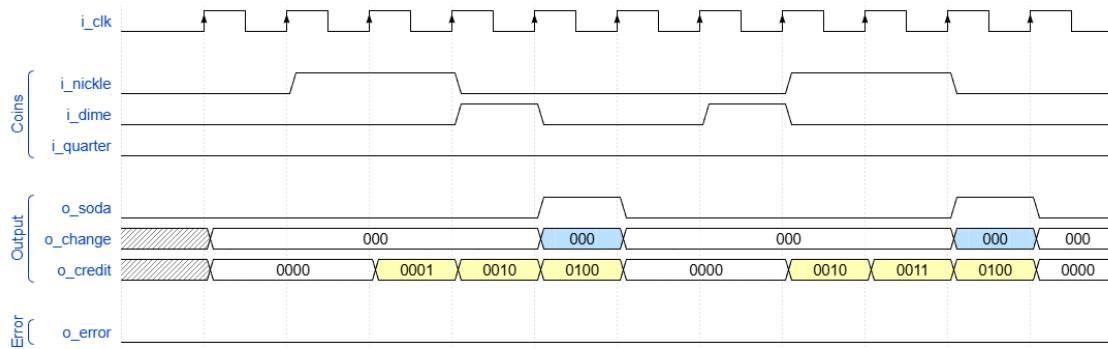


Figure 2: Waveform of an example vending machine receiving coins

In the next example, the system receives a quarter as input. Since the inserted value exceeds the soda price of 20¢, the machine dispenses a soda in the following clock cycle and returns a change of 5¢. While dispensing the soda, the vending machine may still receive additional coins, which will be registered and processed in the next clock cycle.

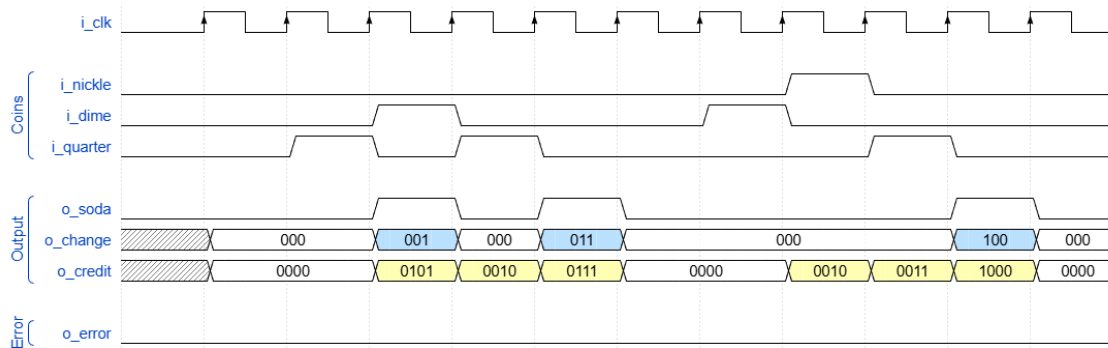


Figure 3: Waveform of an example vending machine returning change

In the final example, the system detects that two or more coin input signals are asserted simultaneously. This condition is considered an error. An error flag is set

high to indicate this faulty behavior. In this case, the vending machine does not dispense a soda, and both the change and credit outputs are set to 0c.

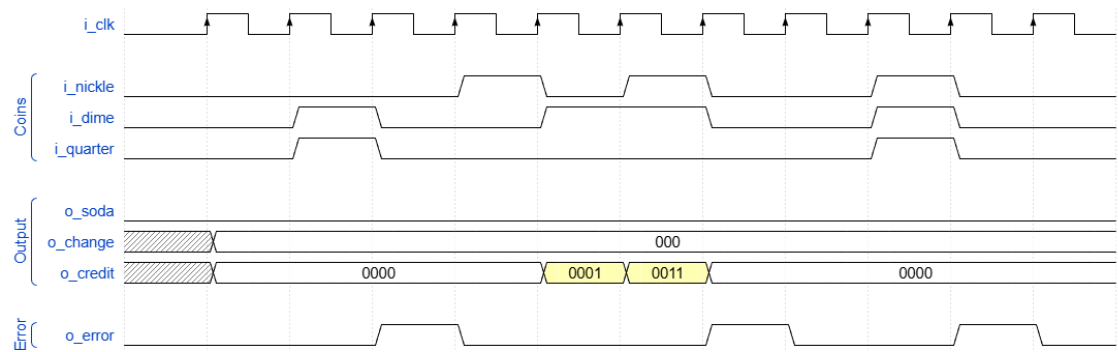


Figure 4: Waveform of an example vending machine with invalid inputs